

# Distributed Cache System Inspired by Redis

## 1. Introduction

This project implements a distributed in-memory cache system inspired by Redis. Its goal is to understand how data can be stored and managed across multiple nodes instead of a single system.

Data is distributed using consistent hashing, where each key is mapped to a node and also stored in a replica node for reliability. This ensures the system continues to work even if a node fails.

The cache supports PUT, GET, and DELETE operations. Writes go to a primary node and are also replicated to another node, while reads first check the primary and fall back to the replica if needed. To manage memory, the system uses TTL (Time-To-Live) for expiration and LRU (Least Recently Used) for removing older entries. When nodes are added or removed, keys are automatically redistributed in the background without interrupting requests.

## 2. System Architecture

### 2.1 High-Level Design

The distributed cache system is composed of four functional layers that work together seamlessly. The client layer accepts incoming requests, the API layer routes them intelligently based on consistent hashing, the cluster management layer coordinates topology changes and failover scenarios, and the cache layer stores data with automatic TTL and LRU management.

Requests flow through the system as follows: clients send PUT, GET, and DELETE operations to the gateway on port 8000. The gateway determines which node owns each key using the consistent hash ring, routing the request to either the primary node or falling back to the replica if the primary is unavailable. The selected node stores or retrieves data from its in-memory cache store. Updates to replicas happen asynchronously in the background to avoid blocking clients. When a node fails, the heartbeat detection system notices within seconds and triggers automatic rebalancing to restore cluster health.

## 2.2 Component Organization

The system organizes into four distinct layers:

- **Client Layer** consists of HTTP clients and Python API callers that send requests to the system. This layer supports both Web integration through the HTTP API and programmatic access for in-process deployments.
- **API Layer** uses FastAPI to route incoming requests intelligently. It implements the consistent hashing algorithm to determine which node owns each key and forwards requests accordingly with automatic failover to replicas.
- **Cluster Layer** manages the overall topology through several key components. The ClusterManager maintains the global view of the ring and node statuses. The Rebalancer coordinates key migration when topology changes occur. The NodeTransport layer abstracts communication details, supporting both in-process and HTTP-based interaction.
- **Cache Layer** implements the actual storage using OrderedDict for LRU tracking and a min-heap for TTL management. Each node runs independently, maintaining its own set of entries up to a configurable maximum size.

## 2.3 Architecture Visualization

The system architecture is captured in the diagram below, showing all five major layers and their interactions:

The diagram illustrates client entry points, the API gateway with routing logic, the cluster management components, three independent cache nodes with replication, and the transport layer enabling distributed communication.

## Implemented Features

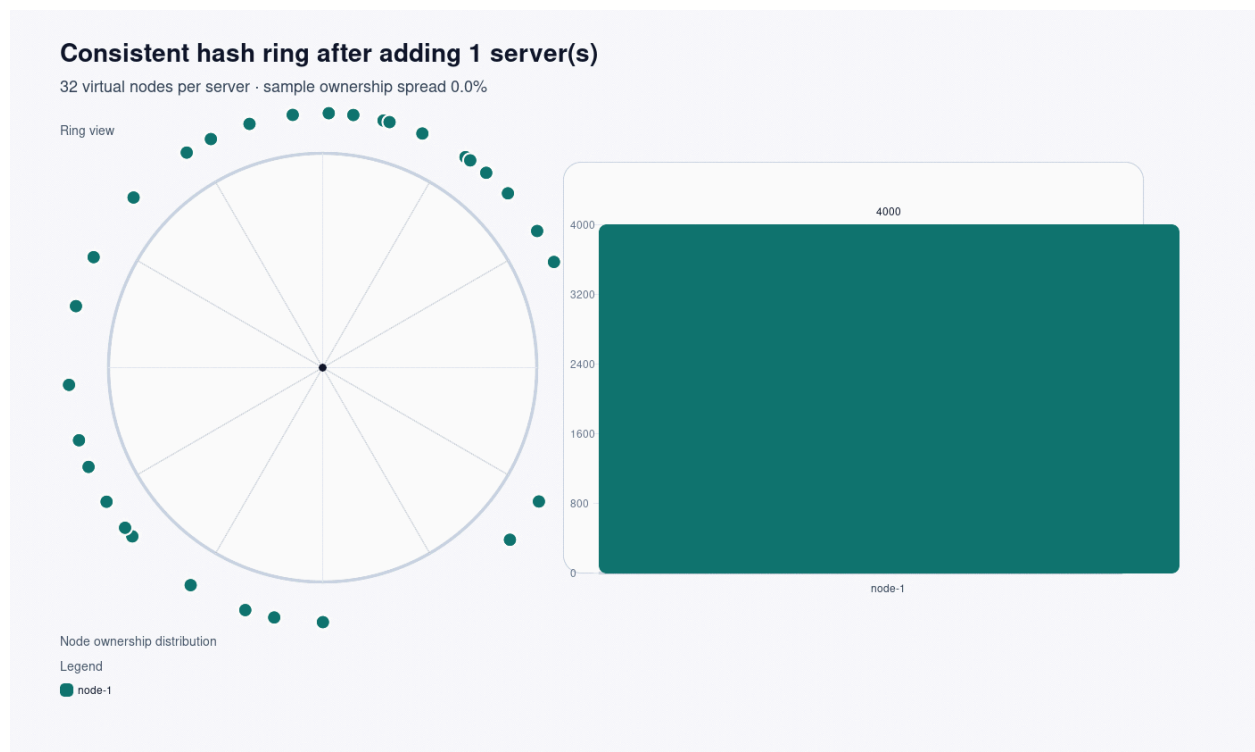
### 1. Consistent Hashing with Virtual Nodes

**How It Works:** The system creates a logical ring from 0 to  $2^{256}$ . Each physical node is represented by multiple virtual nodes (default 32) placed at positions determined by hashing the node ID. When storing a key, the system hashes the key, finds its position on the ring, and uses binary search to locate the next virtual node clockwise—that node becomes the primary owner. The replica is assigned to the second-next virtual node.

**Key Characteristics:**

- SHA-256 hashing ensures deterministic and reproducible key placement
- Virtual nodes (32 per physical node) ensure even distribution across cluster
- Ring recomputation happens automatically when topology changes
- $O(\log n)$  lookup time via binary search on sorted ring positions

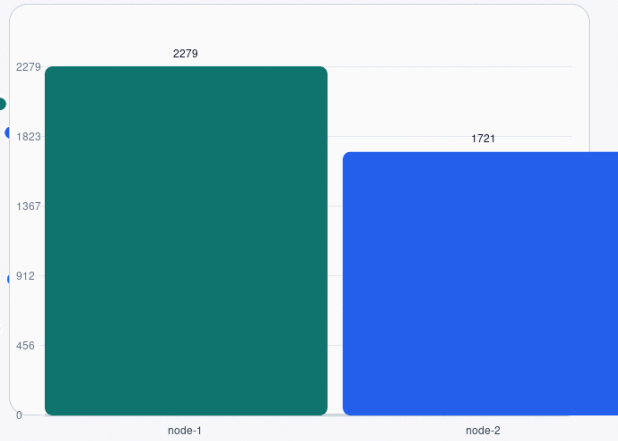
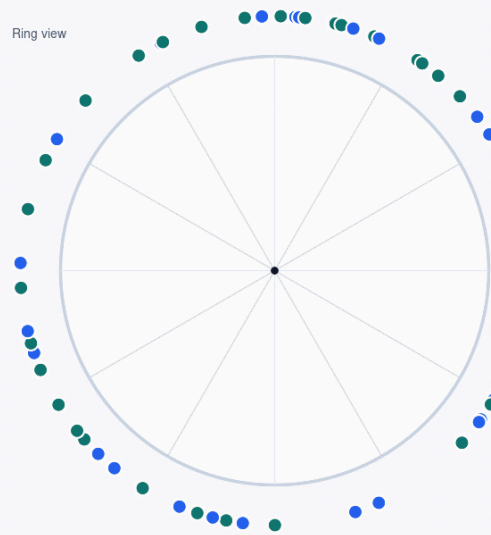
Below Images shows how the consistent hash ring's topology evolves as servers are added one by one. The visualization includes a bar chart showing key distribution across nodes, demonstrating that virtual nodes keep distribution balanced even as the cluster grows.



## Consistent hash ring after adding 2 server(s)

32 virtual nodes per server · sample ownership spread 13.9%

Ring view



Node ownership distribution

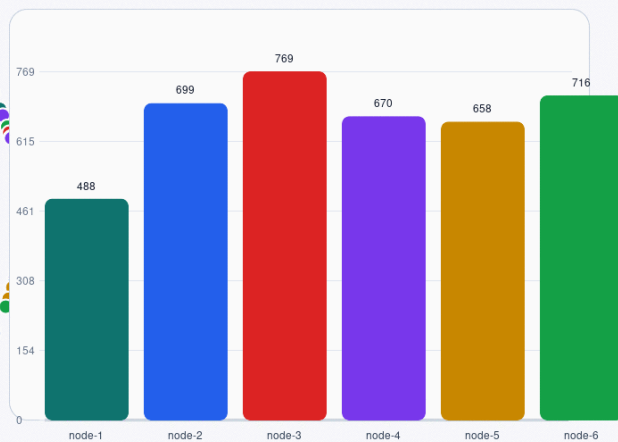
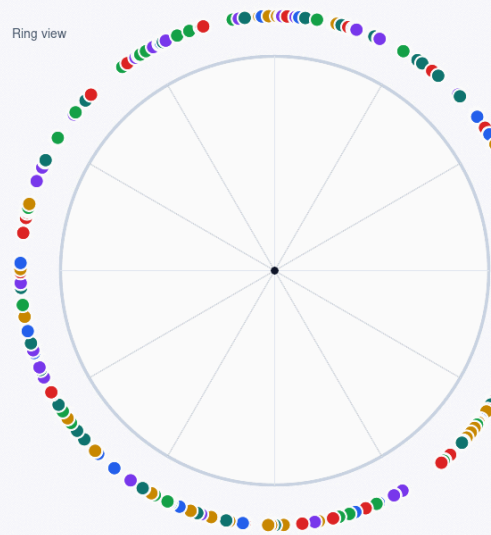
Legend

node-1 node-2

## Consistent hash ring after adding 6 server(s)

32 virtual nodes per server · sample ownership spread 7.0%

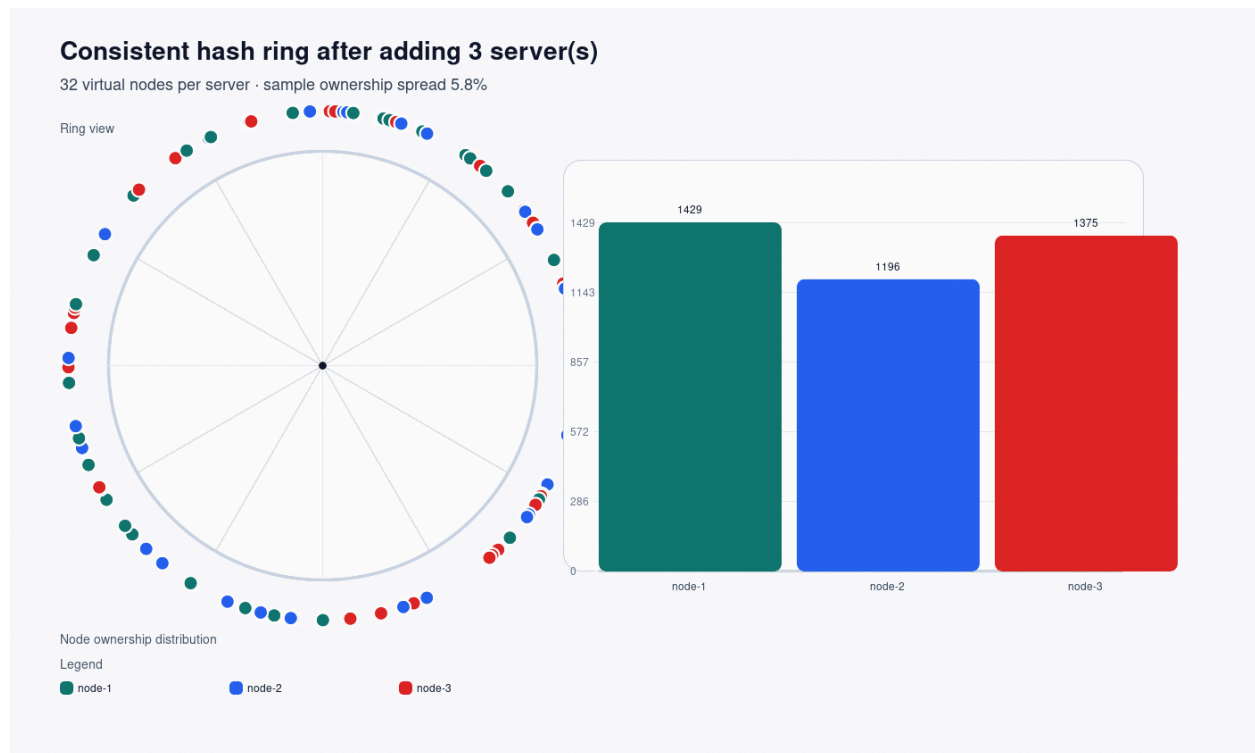
Ring view



Node ownership distribution

Legend

node-1 node-2 node-3  
node-4 node-5 node-6



## 2. Data Replication & Failover

**Implementation:** Every key is stored on both a primary node and a replica node. When a key is written, the primary stores it synchronously (blocking the client until it's stored). The replica update is scheduled asynchronously in the background and does not block the client. On read, if the primary is unavailable, the system automatically serves from the replica.

### Key Characteristics:

- Single replica model (1:1 replication, configurable)
- Automatic replica assignment via ring topology
- Asynchronous replica writes prevent client-side latency
- Replica promotion on primary failure (transparent to clients)
- Heartbeat mechanism detects failures within 3 seconds (configurable)

## 3. TTL (Time-To-Live) Management

**Implementation:** Each key can be stored with a TTL specified in seconds. The store maintains a min-heap of expiration times. When a key is accessed via GET, the system

checks if its expiration time has passed and returns nothing if expired. A background cleanup worker thread periodically wakes up and removes expired entries from the heap and the cache.

**Key Characteristics:**

- Lazy expiration on access (no active removal of unused keys)
- Background cleanup thread removes expired entries periodically
- Min-heap enables efficient expiration tracking
- $O(1)$  insertion,  $O(k)$  cleanup per interval where  $k$  is the number of expired entries

Lazy expiration minimizes unnecessary computation for keys that are never accessed again. The background cleanup thread handles entries that were set with TTL but never accessed again. This approach is much more efficient than actively removing every expired key.

## 4. LRU (Least Recently Used) Eviction

**Implementation:** The cache store uses Python's OrderedDict which maintains insertion order. When a key is accessed, it's moved to the end of the order, marking it as recently used. When the cache reaches its maximum size and a new key must be inserted, the first item (the oldest/least recently used) is automatically removed.

**Key Characteristics:**

- $O(1)$  access, update, and eviction operations via OrderedDict
- Configurable max\_items per node (default 256)
- Automatic eviction on put if cache is full
- No separate index structures needed

The LRU policy ensures the cache never exceeds its configured size limit. Each node on the cluster independently enforces its own limit, allowing the system to scale memory usage linearly with the number of nodes.

## 5. Automatic Rebalancing on Topology Changes

When a node is added or removed, the rebalancing algorithm performs several steps. First, it takes a snapshot of all keys currently on all nodes. Then it iterates through each key to compute its original owner and new owner based on the updated ring. For each

key whose owner changed, it migrates the key to the new owner and deletes it from the old owner. The process runs in the background and handles replication updates as well.

**Key Characteristics:**

- Background rebalancing (non-blocking for client operations)
- Parallel key migration (up to 50 concurrent operations via semaphore)
- Comprehensively handles both node addition and node removal scenarios
- Uses asyncio for non-blocking I/O throughout

## 6. Health Monitoring & Heartbeats

**Implementation:** Each node sends and receives periodic heartbeat messages. The cluster manager tracks the timestamp of the last successful heartbeat for each node. If a node misses heartbeats for a configured duration (default 3 seconds), it's marked as unhealthy and reads fail over to the replica.

**Key Characteristics:**

- Per-node heartbeat timestamps tracked in cluster manager
- Configurable timeout (default 3 seconds)
- Automatic failover when primary is down
- Health report endpoint for monitoring

## Technical Implementation Details

### 1. Consistent Hashing Ring

The consistent hash ring is implemented in ``distributed_cache/cluster/consistent_hash.py`` and maintains several key data structures for efficiency.

The internal data structures include a dictionary mapping hash positions to node addresses, a sorted list of positions for efficient binary searching, a registry of all node configurations, and configuration for the replication factor (virtual nodes per physical node).

**Core operations and their complexities:**

- **Add Node** operation hashes each virtual node and inserts it into the ring, requiring  $O(r \log n)$  time where  $r$  is the replication factor and  $n$  is the current number of ring positions
- **Remove Node** operation identifies and removes all virtual nodes for a specific node, also  $O(r \log n)$
- **Get Node Owner** operation hashes a key and uses binary search to find the next ring position clockwise,  $O(\log n)$
- **Clone Operation** creates a snapshot of the entire ring for rebalancing,  $O(n)$  where  $n$  is the total ring positions

## 2. Cache Store

The cache store is implemented in ``distributed_cache/cache/store.py`` using `OrderedDict` for LRU tracking. It also maintains a min-heap tracking expiration times and uses `RLock` for thread-safe operations.

**All key operations achieve excellent complexity:**

- **Put operations** ( $O(1)$ ) insert or update keys while automatically evicting the least recently used entry if capacity is reached and updating the TTL heap
- **Get operations** ( $O(1)$ ) retrieve values and perform lazy expiration checking, moving accessed keys to the end to mark recency
- **Delete operations** ( $O(1)$ ) remove entries from the cache
- **Snapshot operations** ( $O(m)$ ) iterate all entries to capture state for rebalancing, performing lazy expiration cleanup during iteration
- **TTL cleanup** ( $O(k)$ ) removes expired entries per cleanup cycle where  $k$  is the count of expired entries found

## 3. Cluster Manager & Service

The cluster manager coordinates multi-node operations, implemented in ``distributed_cache/cluster/manager.py`` and ``distributed_cache/cluster/service.py``.

**When a node is added or removed, rebalancing proceeds through distinct phases:**

1. The previous ring state is captured by snapshotting all keys from all nodes

2. The algorithm iterates through all keys to determine whether ownership changed
3. For each key, both the old and new owner (primary and replica) are computed
4. Keys with changed owners are migrated to their new locations
5. Put and delete operations are scheduled asynchronously to new and old owners

Concurrency is carefully managed to prevent overwhelming nodes. A semaphore limits concurrent network operations to 50 simultaneously. Python's asyncio library enables non-blocking I/O throughout, allowing rebalancing to proceed smoothly in the background without blocking client requests.

## 4. Node Transport Layer

The transport layer abstracts node communication, supporting multiple deployment modes.

- **HttpNodeTransport** is used for distributed and Docker deployments. Nodes communicate via HTTP requests using the httpx library. This enables truly distributed scenarios where nodes run on different machines or containers.

- **API Endpoints** exposed by each node include PUT /cache/{key} and GET /cache/{key} for data operations, DELETE /cache/{key} for deletion, POST /internal/heartbeat/{node\_id} for health checks, GET /health for cluster health reporting, and GET /internal/snapshot to retrieve the node's full key inventory during rebalancing.

## Data Structures & Algorithms

### 1. Consistent Hash Ring Algorithm

The core problem in distributed caching is distributing keys evenly when the cluster has  $n$  nodes, such that adding or removing a single node only impacts approximately  $1/n$  of the keys.

**The Algorithm:** The solution uses a virtual node approach on a logical ring. A ring spans 0 to  $2^{256}$  positions. For each physical node, the system creates  $r$  virtual nodes

(default 32) at positions determined by hashing the node ID concatenated with an index. To find a key's owner, the system hashes the key to get a position, then uses binary search to find the next virtual node clockwise on the ring. The replica is assigned to the second-next virtual node.

**Why Virtual Nodes:** Without virtual nodes, a small number of nodes would create a small number of ring positions, leading to uneven distribution and large key movements on topology changes. Virtual nodes spread each physical node across many positions, ensuring even distribution even with small clusters.

**Distribution Property:** With sufficient virtual nodes (32 is typical), the key distribution remains remarkably balanced across the cluster. When a new node is added to  $n$  nodes, approximately  $n/(n+1)$  of the keys stay in place and only  $1/(n+1)$  of the keys move. This property makes the system extremely efficient for dynamic clusters.

## 2. LRU Eviction

LRU tracking uses Python's `OrderedDict` which maintains insertion order as a core data structure property since Python 3.7.

**The algorithm is simple but efficient:**

- When a key is inserted or updated, it's added to or moved to the end of the `OrderedDict` ( $O(1)$  via the `move_to_end()` method)
- When a key is accessed for reading, it's also moved to the end to mark it as recently used ( $O(1)$ )
- When the cache reaches capacity and a new key must be inserted, the first item (oldest/least recently used) is automatically evicted ( $O(1)$ )

**Why This Works:** `OrderedDict`'s `move_to_end()` method provides  $O(1)$  LRU updates without needing separate index structures. Python's implementation uses a doubly-linked list internally, enabling constant-time position changes.

## 3. TTL Expiration Management

TTL management uses a lazy expiration strategy combined with periodic background cleanup.

**On Access:** When a key is retrieved via GET, the system checks whether the current time has exceeded the key's expiration time. If so, the key is treated as missing and not returned to the client.

**Background Cleanup:** A background worker thread maintains a min-heap of (expiration\_time, key\_id) tuples. Periodically, it wakes up and pops expired entries from the heap, deleting them from the cache if they still exist. The cleanup thread prevents the heap from growing unbounded.

**Why Lazy Instead of Active:** Removing every expired key immediately would require expensive operations that scale poorly. Most expired keys may never be accessed again, so removing them only when accessed saves CPU and storage. The background cleanup handles entries that were set with TTL but never accessed.

## Time Complexity Analysis

### 1. Put Operation

**A client putting a key-value pair involves multiple operations:**

1. Router determines the owner node using consistent hashing  $O(\log v)$  where  $v$  is virtual nodes
2. Transport layer delivers the request  $O(1)$
3. Primary node's cache store performs insertion  $O(1)$ , updates TTL heap  $O(\log m)$  where  $m$  is heap size, and potentially evicts an LRU entry  $O(1)$
4. Replica write is scheduled asynchronously  $O(1)$  to enqueue

**Total Complexity:**  $O(\log v + \log m)$ , which is effectively  **$O(1)$  in practice** since  $v$  and  $m$  are small constants.

### 2. Get Operation

**Retrieving a key involves fewer operations than putting:**

1. Router determines owner using consistent hashing  $O(\log v)$
2. Transport layer delivers the request  $O(1)$
3. Storage node's cache store performs dictionary lookup  $O(1)$ , expiration check  $O(1)$ , and LRU update  $O(1)$

**Total Complexity:**  $O(\log v)$ , effectively  **$O(1)$  in practice.**

### 3. Add/Remove Node (Rebalancing)

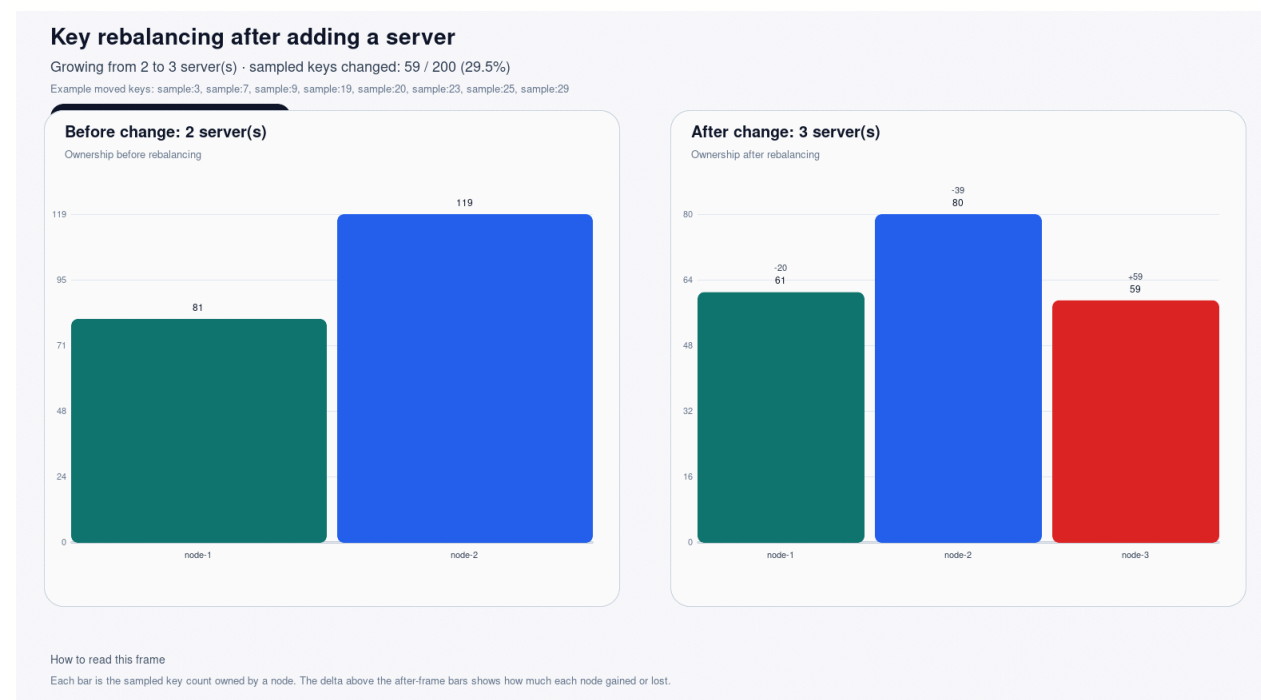
**Adding or removing a node triggers background rebalancing:**

1. Manager adds/removes node from registry  $O(r)$  where  $r$  is replication factor
2. Background rebalancing snapshots all nodes  $O(N*m)$  where  $N$  is nodes and  $m$  is keys per node
3. Algorithm computes owner deltas  $O(m*N)$
4. Keys are migrated in parallel  $O(m')$  where  $m'$  is actually moved keys

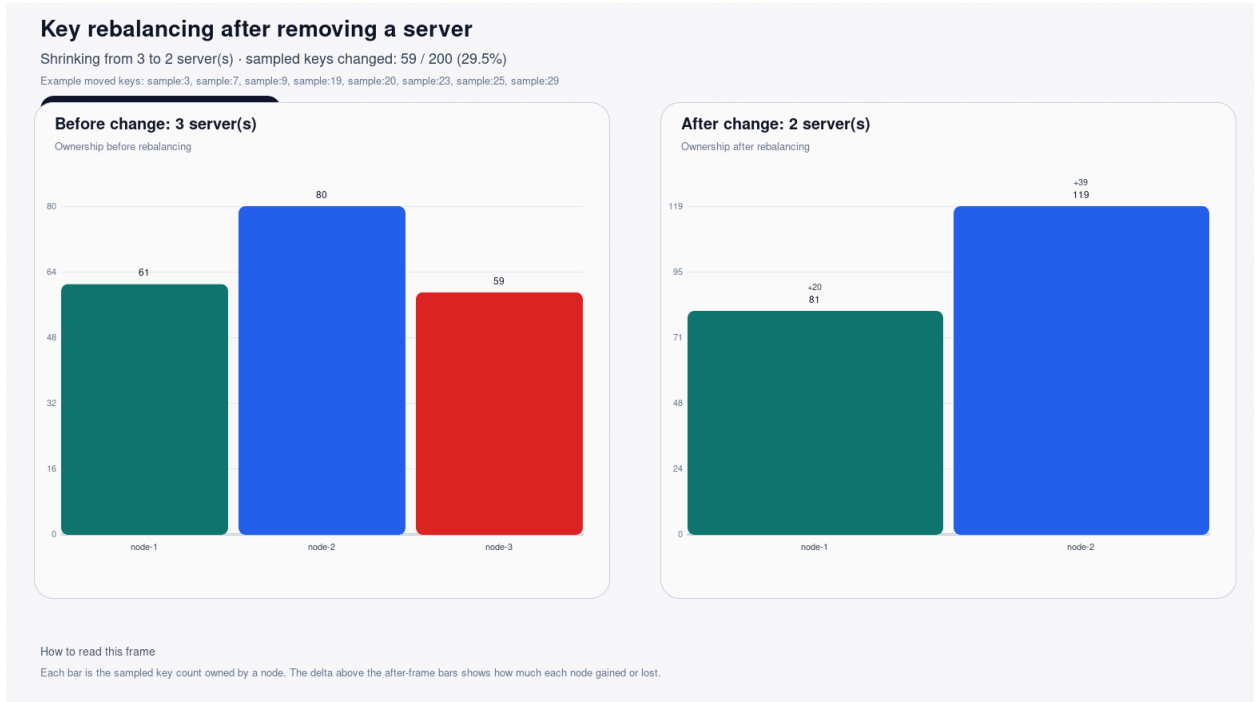
**Theoretical Complexity:**  $O(N*m)$ , but *practical complexity*  $O(m')$  due to parallelization.  
Since only  $\sim 1/n$  of keys move when adding to  $n$  nodes,  $m' \ll N*m$  in typical scenarios.

Below Images compares key ownership before and after topology changes, showing both additions and removals. The Images displays the count of keys that moved and which nodes gained or lost keys.

### Server Addition



# Server Removal



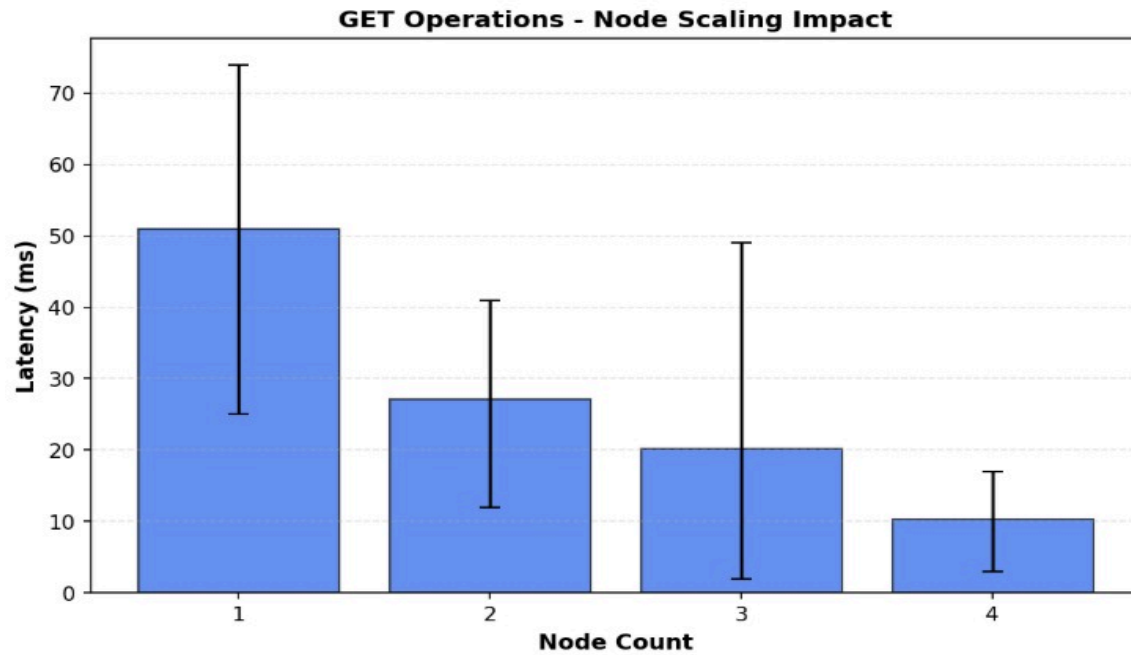
## Metrics :

- Total Operations : 685
- Success Rate : 96.4%

| Operation Type | Count | Successful | Success Rate | Mean Latency (ms) | Throughput (ops/sec) |
|----------------|-------|------------|--------------|-------------------|----------------------|
| GET            | 505   | 480        | 95.0%        | 28.20             | 0.74                 |
| PUT            | 180   | 180        | 100.0%       | 33.95             | 1.00                 |

## Scaling :

- Change in latency on increasing number of Nodes/Servers



- Change in latency to access on increasing number of key-values pairs in the servers.

